

Script Composer Evolution — v1.0 Architecture

Status: Adopted · **Version:** 1.0 · **Owner:** Prasanth (LevelUp AI QA)

Applies to: `levelup-ai-qa-agent` (backend script generation) and its dashboard consumer.

This document is the reference for every Script Composer PR going forward. Cite it in each PR.

0. Purpose

The scenario pipeline is done. Phases A/B (Scenario Data Model + Renderers — PR #247) gave us a single canonical scenario, projected per consumer. We are **not** going to keep redesigning the pipeline.

From here, we make the **existing Script Composer progressively smarter** through deterministic engineering heuristics, ranking algorithms, and quality refinements — never through new orchestrators, intelligence providers, writers, planners, or pipelines.

This document freezes the architecture at **v1.0** and defines the incremental roadmap (Sprints 1, 1.5, 2-7 + the two deterministic rule libraries) that raises generated-code quality without destabilizing the foundation.

1. The Freeze Mandate (read this first)

Freeze the architecture. Do not introduce new orchestrators, intelligence providers, writers, planners, or pipelines. All future improvements must occur inside the existing Script Composer through deterministic engineering heuristics, ranking algorithms, and quality refinements. Every PR must make the generated framework look more like code written by an experienced automation engineer while preserving existing capabilities and avoiding regressions.

The single measure of success

Every future PR is judged by one question:

“Would a senior Playwright engineer approve this pull request without realizing it was AI-generated?”

If a change does not move us toward **yes**, it does not belong in this roadmap.

What this explicitly forbids

- **✗** New orchestrators / intelligence providers / writers / planners / pipelines
- **✗** Moving responsibilities between frozen components (see §2)

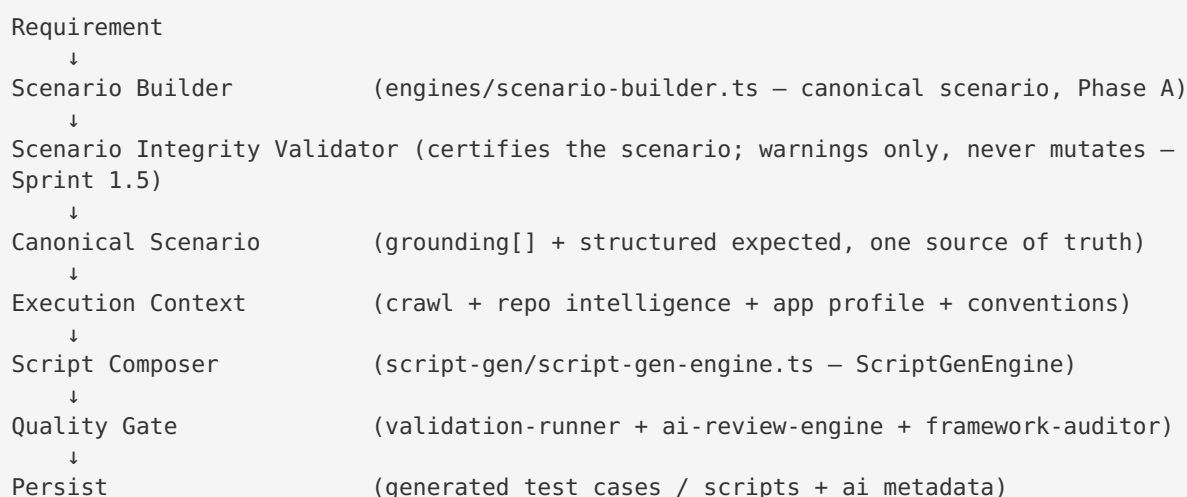
- ❌ More comments, AI explaining every action, verbose logging
- ❌ Huge helper methods, generated documentation, over-engineering
- ❌ Defaulting to the LLM when a deterministic heuristic can decide

What this encourages

- ✅ Better **decisions** inside existing engines (ranking, scoring, fallback)
- ✅ Deterministic heuristics over prompts (no token growth)
- ✅ Reuse of existing framework assets over regeneration
- ✅ Honest confidence signals over pretended certainty

2. The Frozen Pipeline (v1.0)

These are foundation components. **Freeze them. Do not keep moving responsibilities around.**



The **Scenario Integrity Validator** is a deterministic certifier inserted between the Scenario Builder and everything downstream. It does **not** own or mutate scenario data — the Scenario Builder remains the sole owner. It only answers: “Is this canonical scenario internally consistent and automation-ready?” and attaches a readiness score + warnings. See Sprint 1.5.

Where each frozen stage lives in the codebase

Stage	Primary module(s)
Scenario Builder	<code>src/engines/scenario-builder.ts</code>
Scenario Integrity Validator	<code>src/engines/scenario-integrity/</code> (proposed — Sprint 1.5)
Canonical Scenario	<code>DraftTestCase / FormatterTestCase</code> (<code>grounding[]</code> , <code>structured expected</code> , <code>schemaVersion</code>)
Renderers / Projection	<code>src/renderers/scenario-renderer.ts</code> (Manual / Script / BDD)
Execution Context	<code>src/script-gen/page-crawler.ts</code> , <code>src/context/*</code> , <code>src/intelligence/project-convention-profile.ts</code>
Script Composer	<code>src/script-gen/script-gen-engine.ts</code> (<code>ScriptGenEngine</code>)
Quality Gate	<code>src/script-gen/validation-runner.ts</code> , <code>ai-review-engine.ts</code> , <code>framework-auditor.ts</code>
Persist	<code>src/api/routes/script-gen.ts</code> , <code>test-coverage.ts</code>

Everything in the roadmap below happens **inside the Script Composer** and its owned sub-engines. No stage boundary moves.

3. Evolve the Script Composer — not the pipeline

The Script Composer (`ScriptGenEngine`) already composes the sub-engines we will sharpen. We are **not** adding engines; we are making the existing ones decide better.

Sub-engine (exists today)	Role	Sprint that sharpens it
<code>SelectorQualityEngine + intelligence/element-intelligence.rankLocatorCandidates</code>	Locator scoring	Sprint 2
<code>AssertionEngine</code>	Assertions	Sprints 3 & 4
<code>project-convention-profile (buildReuseCatalogue , findReusablePageObject)</code>	Reuse	Sprint 5
<code>ai-review-engine / framework-auditor</code>	Style / audit	Sprint 6
<code>engines/confidence-engine.ts</code>	Confidence	Sprint 7
<code>core/candidate-ranker.ts + core/advisors/*</code>	Ranking (healing side)	Reuse target for Sprint 2

Reuse-first note: the healing side already has a mature scored candidate ranker (`core/candidate-ranker.ts` with `SOURCE_TRUST` , plus `core/advisors/{rule-engine,app-profile,dom-candidate,dom-memory,learned-pattern,ai}-advisor.ts`). Sprint 2 should **reuse and extend** this ranking substrate for generation, not build a parallel one. One ranking brain, two callers.

4. Roadmap

Sprint 1 — Fix & Freeze (this PR set)

-  Fix the **Generate Script** regression (Test Case Lab → Script Gen deep link now selects the requirement, prefills the scenario, and hides/omits App Knowledge because the test case is ready). (dashboard PR #146)
-  Merge PR #247 (Scenario Data Model + Renderers). (owner action)
-  Create this document (`SCRIPT_COMPOSER_EVOLUTION.md`) and **declare the architecture frozen**.
- **No new intelligence.**

Sprint 1.5 — Scenario Integrity Validator ★★★★★ (do before Sprint 2)

A deterministic **certifier** that sits between the Scenario Builder and the Script Composer. It is **not new intelligence** — no LLM, no generation. It answers one question:

“Is this canonical scenario internally consistent and automation-ready?”

Today the Script Composer sometimes compensates for weak test cases. It shouldn't have to. If the Composer always receives a high-quality, certified scenario, locator / assertion / coverage mistakes drop **without touching generation**. This is the first of the two deterministic rule libraries (see §5) and the natural partner to the Engineering Heuristics Library.

It is a Validator, not an Engine. The name is deliberate. Alongside Scenario **Builder**, Script **Composer**, and Quality **Gate**, calling this an “Engine” would imply it owns logic. It does not. It **certifies**. Its entire responsibility is: “Is this scenario internally consistent?” — nothing more.

Hard rule #1: report, never rewrite

Scenario → Validator → Issues → (same) Scenario	✓	
Scenario → Validator → Scenario rewritten	✗	(forbidden)

The Scenario Builder remains the **sole owner** of scenario data. The Integrity Validator only attaches quality signals (warnings + a readiness score); it never mutates steps, grounding, or expected.

Hard rule #2: the readiness score influences confidence, NEVER behavior

The Automation Readiness Score must **never block generation**. Enterprise teams often intentionally automate incomplete requirements — the tool must not become restrictive.

Scenario Ready – 63%	→	Generation blocked.	✗ (forbidden)
Scenario Ready – 63%	→	Generation completed. Confidence: Medium Warnings: - Persona mismatch - Missing grounding	✓

The score influences **confidence and warnings only**. `generationAllowed` is always `true`.

Hard rule #3: it must NOT become a mini Script Composer

The Validator does **not** improve locators, expand assertions, add coverage, or touch the framework. Those belong to the Script Composer (Sprints 2–7). Keep the Validator brutally simple — its only job is internal-consistency certification.

What it validates

Every check below produces **warnings only** — never a hard, generation-blocking failure (Hard rule #2). “warn” means the check found an inconsistency and lowered the readiness score.

- Persona consistency** ★★★★★ — the persona/test data must match the scenario intent.
 - Title: Login with locked user + Test Data: standard_user → **warn**
 - Title: Successful login + Expected: Authentication rejected → **warn**
- Coverage polarity** ★★★★★ — expected outcome must match the coverage type (deterministic):
 - positive → success state (e.g. dashboard displayed)
 - negative → error shown
 - edge → graceful validation
 - boundary → limit accepted/rejected

3. **Test-data suitability** ★★★★★ — dataset must fit the scenario.
 - valid login scenario + locked_user dataset → **warn**
4. **Expected-result consistency** ★★★★★ — manual outcome vs. structured expected must agree.
 - manual Login successful + expected Error displayed → **warn**
5. **Step completeness** ★★★★★ — steps must plausibly produce the expected outcome.
 - Click Login → expected Inventory page , but no username/password entered → **warn (incomplete)**
6. **Missing preconditions** ★★★★★ — required preconditions must be present.
 - Checkout without User logged in → **warn**
7. **Business flow consistency** ★★★★★ — steps must follow a possible state progression.
 Deterministic state-order validation (no AI): each step maps to a business stage
 (open → login → browse → add-to-cart → cart → checkout → payment → confirmation → logout),
 and impossible orderings are flagged.
 - Checkout → Payment → Add item → **warn** (add-to-cart after checkout is impossible)
 - Logout → Add item → **warn** (authenticated action after logout is impossible)
 - Checkout with no prior Add item → **warn** (checkout without a cart)
8. **Grounding completeness** ★★★★★☆ — every actionable step should have grounding.
 - Enter username with no grounding → **do not warn hard; reduce confidence** (score penalty only).

Automation Readiness Score

Before the Script Composer starts, the Validator computes a scenario-quality score so the Composer knows

how much to trust the scenario (confidence only — it never blocks, per Hard rule #2):

Check	Result
Persona	✓
Coverage polarity	✓
Test Data	✓
Expected	✓
Step completeness	✓
Preconditions	✓
Business flow	✓
Grounding	⚠

Scenario Ready — 96% · Confidence: High

The score (and the per-check breakdown) is carried alongside the scenario and later feeds the Sprint 7 Confidence Engine and Smart TODOs — honest signal instead of silent compensation.

One number is not enough — expose dimensions to the Quality Gate

A single readiness number isn't actionable. The Validator's score is the **Scenario Quality** dimension; the Quality Gate combines it with the Composer's own quality signals into one report:

Scenario Quality	95	(Scenario Integrity Validator – this sprint)
Framework Quality	97	(framework-auditor)
Locator Quality	94	(Sprint 2)
Assertion Quality	96	(Sprints 3 & 4)
Overall Ready	95	

The Validator owns only **Scenario Quality**; the other dimensions are filled in by later sprints. The report shape is designed so each dimension plugs in without changing the others.

Where it lives

`src/engines/scenario-integrity/` — a pure, dependency-light rule module. Output is a typed `ScenarioIntegrityReport { readinessScore, confidence, generationAllowed, checks[], warnings[] }`

attached to the scenario (persisted read-only in `ai_metadata.scenarioIntegrity`). No stage boundary moves; the Scenario Builder still owns the data. `generationAllowed` is **always true**.

Milestones (ship small — measure after each):

- **1.5a** — Report scaffold + persona / expected-result / coverage-polarity checks (the three highest-signal, purely-textual checks).
- **1.5b** — Test-data suitability + missing-preconditions + step-completeness checks.
- **1.5c** — Business-flow consistency (deterministic state-order) + grounding-completeness (confidence penalty).
- **1.5d** — Automation Readiness Score aggregation + Scenario Quality dimension surfaced to the Quality Gate & dashboard (read-only, never blocking).

Sequencing: only after the Scenario Integrity Validator is stable do we begin Sprint 2.

Sprint 2 — Locator Ranking Engine ★★★★★ (highest ROI)

This is **not** “better locators.” It is **better locator decision-making**.

Today

Grounding → Selector

Target

Grounding → Candidate Discovery → Candidate Ranking → Candidate Selection → Confidence

4.1 Candidate Discovery — collect, do not select

Gather candidates from every source, in priority order. **Do not choose yet.**

1. Existing **Page Object** methods (reuse wins)
2. **App Profile** (crawled grounding)

3. **Test Case** wording
4. **Expected Result** text
5. **Requirement** text
6. **DOM** relationships
7. **Accessibility** (role + name, aria-label, label association)
8. **AI** — last resort only

Fallback search chain before AI. When the App Profile misses an element, do **not** jump to the LLM. Search deterministically first:

Test Case → Keywords → Expected Result → Requirement → Page Object → DOM → AI

Example — manual step `Click Login`, expected `Dashboard` appears : the composer infers Login causes navigation and searches for `login / sign in / submit / authenticate` candidates instead of hallucinating a selector.

4.2 Candidate Ranking — score every candidate

Reuse/extend `core/candidate-ranker.ts` scoring. Indicative scores:

Source / strategy	Score
Existing Page Object method	100
<code>data-testid</code>	98
<code>role</code> + accessible name	96
<code>aria-label</code>	95
label association	94
placeholder	88
stable CSS	82
DOM fingerprint	78
XPath	40

Never just use the first locator. Find → Rank → Choose.

4.3 Candidate Selection — score the strategy, and record why

- Choose the highest-scoring candidate.
- If the top two are close, keep **both** — record the runner-up as an alternative.
- Score the **locator strategy**, not just the string — and persist the **reasons**. Debuggability and auditability come for free: when someone asks “why didn’t LevelUp use XPath?”, the answer is already stored.

Chosen: `getByRole('button', { name: 'Login' })` strategy: `role` score: 96

Why: `✓ accessible ✓ stable ✓ user-facing ✓ matches framework convention`

Alternative: `getByTestId('login-btn')` strategy: `testid` score: 98

(framework prefers role)

4.4 Confidence banding (feeds Sprint 7)

Confidence	Behavior
> 90	Generate normally.
70-90	Generate normally and record Review recommended.
< 70	Generate a TODO with a reason (e.g. Dynamic element) and a suggested locator.

Milestones (ship small — measure after each):

- **2a** — Candidate Discovery: collect candidates from all sources into a typed list (no behavior change to selection yet).
- **2b** — Ranking: score candidates via the shared ranker; log chosen vs. alternatives.
- **2c** — Selection + alternative retention.
- **2d** — Confidence banding + TODO emission for < 70 .

Sprint 3 — Assertion Expansion ★★★★★

Automation should validate **more** than the manual test, because it can. Deterministic expansion inside `AssertionEngine` — **not** prompting.

Input `Verify login successful` → **Output**

- ☒ URL changed
- ☒ Dashboard visible
- ☒ Logout button visible
- ☒ Session established
- ☒ No error present
- ☒ Inventory / landing content loaded

Rules: verify the **business outcome first**, then add one or two **stability** assertions. Never generate redundant assertions.

Sprint 4 — Coverage Expansion ★★★★★

Different from Sprint 3. Not more assertions — **more business coverage**.

Manual `Add item to cart` → **Automation additionally verifies**

- cart badge updated
- item count
- total price
- persistence (survives reload / navigation)
- navigation correctness

Meaningful, business-relevant checks only — never random assertions. Automation coverage should **always exceed** the manual test case.

Sprint 5 — Framework Reuse ★★★★★

Improve **reuse**, not generation. Aggressively consult

`project-convention-profile ( buildReuseCatalogue , findReusablePageObject )` and repo pattern analysis.

```
login() ; login() ; login()    →    authenticatedFixture()
```

If `login()` (or any page-object method / fixture / helper) already exists, **reuse it — never regenerate**. The less code we generate, the more senior it looks.

Sprint 6 — Humanization ★★★★★

The framework must not feel generated. The problem is rarely correctness — it is that the output is **too symmetrical**. Senior engineers extract methods, simplify, and drop noise.

Reduce, inside `ai-review-engine / framework-auditor`:

- repetition
- unnecessary comments
- unnecessary variables
- over-defensive code
- identical naming everywhere

Make it feel like a teammate wrote it — following project conventions.

Sprint 7 — Confidence Engine ★★★★★

Last. Do not rewrite. Do not regenerate. Simply identify uncertainty and be honest about it, building on `engines/confidence-engine.ts` and the Sprint 2 bands.

Before returning, the composer asks: **“Would I merge this PR?”** — checking locator, assertion, reuse, compile, and framework confidence. When confidence is low, **flag**, don’t guess.

Smart TODOs (not AI noise)

Emit TODOs **only** when confidence is genuinely low — never a wall of `TODO TODO TODO`.

```
// Review locator:  
// Dynamic element detected after login.  
// Suggested locator:  
// page.getByRole('button', { name: /checkout/i })
```

5. The two deterministic rule libraries (the secret sauce)

The system will have **exactly two** deterministic rule libraries — and no third intelligence layer. They complement each other without adding an AI component or growing prompts:

Library	Question it answers	Stage
Scenario Integrity Validator (§4, Sprint 1.5)	“Is the canonical scenario internally correct and automation-ready?”	Before the Composer
Engineering Heuristics Library (below)	“Given a correct scenario, what is the best way to implement it as production-quality automation?”	Inside the Composer

Together they give a stronger long-term foundation than continually expanding prompts or adding new AI components: one certifies the **input**, the other governs the **implementation** — both deterministic, both token-free, both versioned and unit-tested.

5.1 Engineering Heuristics Library

Not another intelligence. A **deterministic rule library** that captures how experienced automation engineers think. Every generation consults it. **No LLM. No prompt. No tokens.** It improves the product over time without growing prompt size.

Proposed home: `src/script-gen/heuristics/` (a pure, dependency-light rule module consumed by `ScriptGenEngine`). Rules are data + pure functions — unit-tested and versioned.

Rule families

Locator heuristics

- Prefer existing page-object methods.
- Prefer `data-testid`.
- Prefer accessible roles and names.
- Avoid brittle CSS / XPath unless necessary.

Assertion heuristics

- Verify business outcome first.
- Add one or two stability assertions.
- Don't generate redundant assertions.

Framework heuristics

- Reuse existing fixtures.
- Reuse page-object methods.
- Don't create duplicate utilities.

Naming heuristics

- Follow the project's existing naming conventions.
- Avoid identical/templated names everywhere.

Playwright heuristics

- No `waitForTimeout` ; use web-first assertions and auto-waiting.
- Prefer user-facing locators (`getByRole` / `getByLabel`).

These heuristics evolve over time (Sprint 7+ “continuous learning” improves the weights from successful generations) — **without** increasing token usage or architectural complexity.

6. Long-term stable roadmap

Phase	Goal	Change type
1	Fix & Freeze	Regression fix + freeze declaration
1.5	Scenario Integrity Validator	Certify the scenario (readiness score + warnings); never mutate
2	Locator Ranking Engine	Improve selection , no architecture changes
3	Assertion Expansion	Increase automation coverage
4	Coverage Expansion	Add meaningful automation-only validations
5	Framework Reuse	Reduce duplication
6	Human-like Code Style	Output indistinguishable from hand-written code
7	Confidence Engine	Flag uncertain areas instead of guessing
(ongoing)	Continuous Learning	Improve heuristic weights from successful generations

Notice what is missing: no new orchestrators, no new intelligence providers, no new pipelines, no major rewrites.

7. PR checklist (paste into every Script Composer PR)

- [] Change lives **inside** the Script Composer / its owned sub-engines — no new orchestrator/provider/writer/planner/pipeline, no stage boundary moved.
- [] Prefers a **deterministic heuristic** over an LLM call where a decision can be made.

- [] Reuses existing framework assets (page objects, fixtures, helpers) before generating.
- [] Adds **no** unnecessary comments / logging / variables / defensive code.
- [] Preserves existing capabilities (no regressions); tests updated.
- [] Emits honest confidence / Smart TODOs instead of pretending certainty.
- [] Answers **yes** to: “Would a senior Playwright engineer merge this without realizing it was AI-generated?”
- [] References this document (`SCRIPT_COMPOSER_EVOLUTION.md`).

8. Appendix — component map (as of v1.0)

<code>src/engines/scenario-builder.ts</code>	canonical scenario (Phase A)
<code>src/engines/scenario-integrity/</code> (proposed)	Scenario Integrity Validator → Sprint 1.5 / \$5
<code>src/renderers/scenario-renderer.ts</code>	Manual / Script / BDD projection (Phase B)
<code>src/script-gen/script-gen-engine.ts</code>	ScriptGenEngine – the Script Composer
└─ <code>selector-quality-engine.ts</code>	locator scoring → Sprint 2
└─ <code>assertion-engine.ts</code>	assertions → Sprints 3 & 4
└─ <code>wait-strategy-engine.ts</code>	smart waits
└─ <code>workflow-mapper.ts</code>	navigation graph & flows
└─ <code>validation-runner.ts</code>	compile / validate (Quality Gate)
└─ <code>ai-review-engine.ts</code>	style review → Sprint 6
└─ <code>framework-auditor.ts</code>	framework audit → Sprint 6
<code>src/intelligence/element-intelligence.ts</code>	<code>rankLocatorCandidates</code> → Sprint 2
<code>src/intelligence/project-convention-profile.ts</code>	reuse catalogue → Sprint 5
<code>src/core/candidate-ranker.ts</code>	scored ranker (reuse) → Sprint 2
<code>src/core/advisors/</code> *	ranking advisors (reuse) → Sprint 2
<code>src/engines/confidence-engine.ts</code>	confidence → Sprint 7
<code>src/script-gen/heuristics/</code> (proposed)	Engineering Heuristics Library → \$5